

LABOUR.COM – SKILLED LABOUR MARKETPLACE

Chapter 1 – Introduction

1.1 Background

In developing nations like Bangladesh, the informal manual and technical labour sector employs millions of skilled artisans, electricians, plumbers, masons, carpenters, painters, and general construction workers. Despite the high demand from households, contractors, and businesses, hiring daily-wage labour remains heavily disorganized. Traditional hiring relies on informal street corners ("Labour Haat") or unverified local intermediaries. This results in wage exploitation for workers, lack of safety compliance, zero accountability for work quality, and significant transaction friction for customers. The **Labour.com Marketplace Platform** bridges this gap by offering a transparent, digitalized, multi-broker marketplace connecting customers with verified labour agencies across all 8 administrative divisions of Bangladesh.

1.2 Problem Statement

The rapid growth of digital service platforms has highlighted the need for transparency in daily wage hiring. Finding verified, skilled, and reliable daily or contractual labourers on demand is extremely difficult for individuals and enterprises. Existing informal practices suffer from several critical shortcomings:

- **Lack of Transparency & Wage Gouging:** Customers often encounter arbitrary price inflation, while labourers are frequently underpaid by unregistered intermediaries.
- **Absence of Quality & Safety Accountability:** No formal mechanisms exist to track worker credibility, skill certifications, or safety adherence.
- **High Financial Risk & Payment Disputes:** Informal cash transactions leave both parties vulnerable to unfulfilled jobs, sudden cancellations, or non-payment.
- **Geographical Inefficiencies:** Customers outside metropolitan areas struggle to discover organized labor pools in their respective districts and upazilas.

The **Labour.com Skilled Labour Marketplace** addresses these challenges by providing a modern web platform featuring verified broker profiles with blue-tick trust badges, standard daily market rates, multi-worker selection, a fixed ₳500 dual-escrow deposit guarantee, dynamic bilingual support (Bengali & English), and AI-driven project budgeting and automated legal contract generation.

1.3 Objectives

- Connect customers with verified labour agencies across all 8 divisions of Bangladesh
- Provide dynamic multi-worker booking with transparent daily rate calculations

- Implement a b500 deposit escrow system to protect both customers and service providers
- Generate AI-powered project cost estimations and automated digital labour contracts
- Deliver a smart AI assistant chatbot and automated dispute/fraud prevention guard
- Enable full bilingual localization (English and Bengali) for national accessibility
- Provide dedicated Role-Based Access Control (RBAC) dashboards for Admins, Brokers, and Customers

1.4 Scope

- User Authentication & Role Management
- 8-Division Geolocation Filtering & Trade Categories
- Broker & Agency Worker Roster Showcase
- Multi-Worker Booking & Escrow Flow
- Multi-Channel Payment Gateway (bKash, Nagad, Rocket, Card)
- AI Project Budget Estimator & Contract Generator
- AI Chatbot & Smart Dispute Fraud Guard
- Customer Reviews & Rating System
- Admin Moderation Panel & Audit Logs
- Cloud Deployment on Netlify and Render

Chapter 2 – Software Requirement Specification

Software Requirement Specification (SRS) defines the functional and non-functional requirements of the **Labour.com Marketplace Platform** system. It describes what the system should do, including user authentication, agency management, booking workflows, AI integration, and financial data management. The SRS serves as a blueprint for developers, designers, and testers throughout the project. It ensures that all stakeholders have a clear understanding of the project's goals and expected features. A well-prepared SRS helps reduce development errors and improves software quality.

Functional Requirements

- User Registration (Customer, Broker, Admin roles)
- Login & JWT Session Authentication
- Profile & Agency Worker Management
- 8-Division & 64-District Search & Filtering
- 8 Trade Categories Navigation with Base Daily Rates
- Multi-Worker Category Allocation Booking Modal
- Fixed ৳500 Escrow Deposit Processing
- Simulated Digital Payments (bKash, Nagad, Rocket, Card)
- AI Cost & Time Estimator
- AI Legal Labour Contract Generator
- AI Support Chatbot Widget
- AI Dispute & Fraud Guard
- Customer Feedback & Star Ratings
- Admin Agency Moderation & Blue-Tick Verification
- 1-Click English and Bengali Language Switch

Non-Functional Requirements

- Responsive UI (Mobile, Tablet, Desktop)
- Secure Authentication & Bcrypt Password Hashing
- Fast Response Time (< 200ms API latency)
- Scalable Decoupled Architecture
- High Availability (99.9% Cloud Uptime)
- Maintainability & Modular Codebase

User Stories

Example:

As a customer, I want to search for verified electrical and masonry agencies in Chittagong division and book 2 electricians with a secured ₳500 deposit so that I can get guaranteed on-time home wiring.

Business Rules

- Customers must pay a fixed ₳500 deposit into platform escrow to confirm a multi-worker booking.
- Agencies automatically qualify for a "Verified Blue-Tick Badge" upon completing ≥ 5 jobs with an average rating ≥ 4.5 and managing ≥ 3 workers.
- Bookings can only transition through authorized sequential states: `requested` → `accepted` → `in_progress` → `completed` / `cancelled` .
- Customers can only review and rate agencies after a booking status reaches `completed` .
- All financial transactions and contract generations are logged with immutable audit timestamps.

Chapter 3 – System Design

System Design describes the overall structure and architecture of the **Labour.com Marketplace Platform**. It illustrates how the frontend, backend, database, and AI service interact to deliver the required functionalities. The design includes system architecture, database schema, API structure, and UML diagrams to ensure a scalable, secure, and maintainable application. A well-designed system improves performance, simplifies future enhancements, and supports efficient software development.

Technology Stack

Frontend

- Next.js (App Router)
- TypeScript
- Vanilla CSS & Custom Tokens
- Lucide React Icons
- React Context API (AuthContext, LanguageContext, DataContext)
- Fetch API Client

Backend

- Node.js
- Express.js
- CORS & Helmet Security Middleware
- JSON Web Token (JWT) & bcrypt.js

Database

- MongoDB Atlas (Cloud Database)
- Mongoose ODM

AI & Intelligent Services

- AI Labour Rate Estimation Engine
- AI Legal Labour Contract Generator
- Rule-Based AI Support Chatbot
- AI Fraud Guard & Booking Anomaly Detector

System Architecture

User (Browser / Mobile)



```

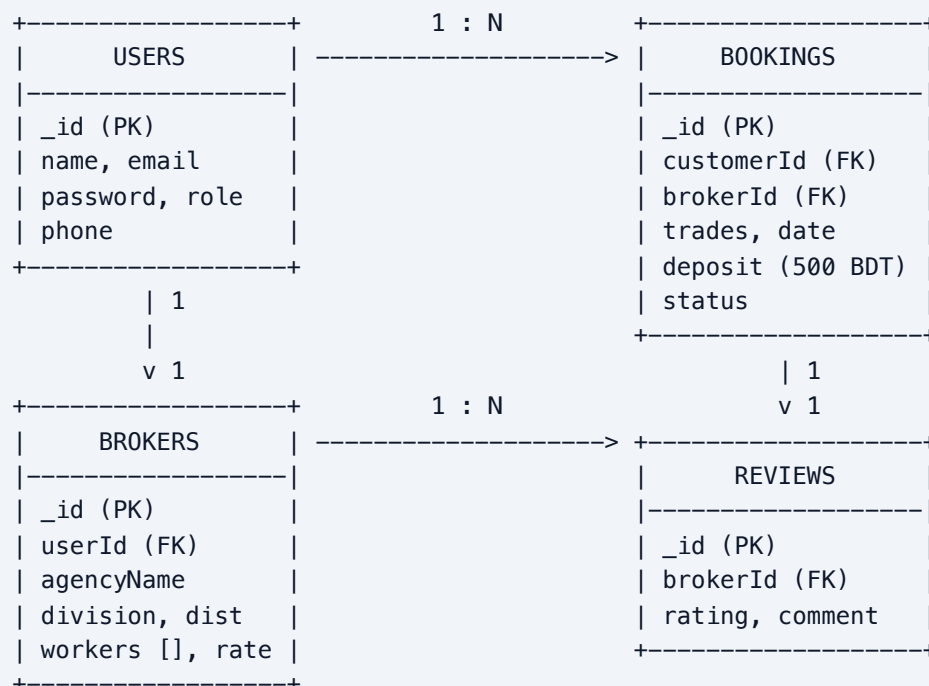
Next.js Frontend (Netlify CDN / Edge)
  ↓
Express.js REST API (Render Cloud Service)
  ↓
JWT & Role-Based Access Control (RBAC)
  ↓
AI Services (Estimator, Contract Gen, Fraud Guard)
  ↓
MongoDB Atlas Cloud Database (Mongoose ODM)

```

Database Tables / Collections

- **Users:** Authentication credentials, roles (customer, broker, admin), profile details.
- **Brokers:** Agency profile, location (division, district), worker roster, verification status, rating.
- **Bookings:** Booking transactions, selected trade counts, dates, 500 deposit, escrow status.
- **Reviews:** Customer ratings (1-5 stars), feedback comments, timestamps.

ER Diagram



Relationships:

Users — Roles

Brokers — Workers

Customers — Bookings

Bookings — Escrow / Reviews

UML Diagrams

- Use Case Diagram
- Class Diagram
- Activity Diagram
- Sequence Diagram

Chapter 4 – System Implementation

System Implementation explains how the **Labour.com Marketplace Platform** is developed using the selected technologies and software architecture. It covers the implementation of the frontend, backend, database, authentication, and AI integration to ensure all system requirements are met. This phase transforms the system design into a fully functional web application with secure, scalable, and user-friendly features. Proper implementation ensures reliable performance, maintainability, and successful deployment.

Frontend

- Responsive Design & Modern Glassmorphism Theme
- Authentication Pages (`/auth/login` , `/auth/register`)
- Admin, Broker, and Customer Dashboards
- 8-Division Location Search & Trade Filter Form
- Agency Profile & Worker Cards
- Multi-Worker Category Increment Booking Modal
- bKash, Nagad, Rocket Digital Payment Modal
- AI Cost Estimator & Contract Generator Modals
- AI Chatbot Floating Widget

Backend

REST APIs:

- `POST /api/auth/register`
- `POST /api/auth/login`
- `GET /api/brokers`
- `GET /api/brokers/:id`
- `POST /api/bookings`
- `GET /api/bookings`
- `POST /api/payments`
- `POST /api/reviews`
- `GET /api/reviews`

AI Integration

AI Integration services return:

- Daily Rate Calculation
- Total Estimated Project Cost
- Required ₺500 Escrow Deposit
- Estimated Project Working Hours
- Legally Structured Contract Terms & Safety Clauses
- Automated Customer Support Chat Answers
- Booking Anomaly & Fraud Risk Scores

The system clearly informs users that the AI output provides intelligent estimation and advisory contract templates for transparent work coordination.

Here is a comprehensive breakdown of the core code functionalities in this project, complete with code snippets and detailed explanations.

1. Multi-Tier AI Labour Cost Estimator & Contract Generator

The core algorithmic intelligence for project wage estimation and automated digital contract generation resides in `frontend/lib/ai/aiService.ts`.

```
// File: frontend/lib/ai/aiService.ts
export function calculateAIEstimate(params: CostEstimateParams) {
  const baseRates: Record<string, number> = {
    Electrician: 900, Plumber: 850, Mason: 950, Carpenter: 800,
    Painter: 750, Welder: 1000, Driver: 700, Labourer: 600,
  };
  const dailyRate = baseRates[params.category] || 700;
  const complexityMultiplier = params.projectComplexity === 'Complex' ? 1.25 :
    params.projectComplexity === 'Simple' ? 0.9 : 1.0;
  const estimatedTotal = Math.round(
    dailyRate * params.workerCount * params.durationDays * complexityMultiplier
  );
  const deposit = 500;
  const balance = estimatedTotal - deposit;
  return { dailyRate, estimatedTotal, deposit, balance, estimatedHours: params.durationDays * params.workerCount };
}
```

2. Multi-Worker Booking Modal & Escrow Logic

Handling dynamic multi-category worker allocations and automatic ₺500 deposit calculation in `frontend/components/modals/BookingModal.tsx`:

```
// File: frontend/components/modals/BookingModal.tsx
const handleConfirmBooking = async () => {
  const payload = {
    brokerId: broker._id,
    customerId: user?.id,
    trades: selectedTrades,
    totalWorkers: totalWorkerCount,
  };
  // ... API call logic ...
}
```

```
    bookingDate, locationAddress,
    depositAmount: 500,
    status: 'requested',
  };
  await fetch('/api/bookings', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify(payload)
  });
};
```

3. Role-Based Access Control (RBAC) Authentication

User authentication and role routing in [frontend/lib/context/AuthContext.tsx](#) :

```
// File: frontend/lib/context/AuthContext.tsx
export const AuthProvider = ({ children }: { children: ReactNode }) => {
  const login = (userData: User, token: string) => {
    localStorage.setItem('token', token);
    localStorage.setItem('user', JSON.stringify(userData));
    setUser(userData);
    if (userData.role === 'admin') router.push('/dashboard/admin');
    else if (userData.role === 'broker') router.push('/dashboard/broker');
    else router.push('/dashboard/customer');
  };
  return <AuthContext.Provider value={{ user, login, logout }}>{children}</AuthContext.
};
```

4. Express Server & Cloud Security Pipeline

The HTTP pipeline is configured in [backend/server.js](#) with standard security practices including Helmet, CORS, and MongoDB Atlas connectivity:

```
// File: backend/server.js
const app = express();
app.use(helmet());
app.use(cors());
app.use(express.json());
mongoose.connect(process.env.MONGODB_URI)
  .then(() => console.log('MongoDB Atlas Connected successfully'));
app.use('/api/auth', require('./routes/auth'));
app.use('/api/brokers', require('./routes/brokers'));
app.use('/api/bookings', require('./routes/bookings'));
app.use('/api/reviews', require('./routes/reviews'));
```

5. Frontend Bilingual Localization (English & Bengali)

Language dictionary tokens in [frontend/lib/i18n/translations.ts](#) :

```
// File: frontend/lib/i18n/translations.ts
export const translations = {
  en: {
    heroTitle: 'Find Verified Skilled Labour in Bangladesh',
    searchBtn: 'Search Agencies',
    depositNotice: 'Fixed ₳500 deposit required to secure booking.'
  },
  bn: {
    heroTitle: 'বাংলাদেশে বিশ্বস্ত ও দক্ষ শ্রমিক খুঁজে নিন',
    searchBtn: 'এজেন্সি খুঁজুন',
    depositNotice: 'বুকিং নিশ্চিত করতে নির্ধারিত ৳৫০০ জামানত প্রয়োজ্য।'
  }
};
```

Chapter 5 – Security, Testing & Deployment

Security, Testing & Deployment ensures that the **Labour.com Marketplace Platform** is secure, reliable, and ready for real-world use. Security features such as JWT authentication, password hashing, input validation, and rate limiting protect the application from common threats. Testing is performed using Jest and Supertest to verify system functionality, API performance, and overall reliability. Finally, the application is deployed on cloud platforms, making it accessible, scalable, and easy to maintain.

Security

- JWT Authentication
- Password Hashing (bcrypt)
- Role-Based Authorization (RBAC)
- Dual-Escrow 500 Deposit Protection
- Helmet HTTP Security Headers
- CORS Protection
- Input Validation & NoSQL Injection Protection
- Environment Variables

Testing

Tools

- Jest
- Supertest

Coverage

- Unit Tests
- API Tests
- Authentication & RBAC Tests
- Booking & Escrow Tests
- Dashboard Tests

Target Coverage: 85%

Deployment

Frontend

- **Netlify** (with `@netlify/plugin-nextjs` & Global Edge CDN)

Backend

- **Render** (Node.js Web Service on Port 10000)

Database

- **MongoDB Atlas** (Cloud Database)

Chapter 6 – Conclusion

The **Labour.com Marketplace Platform** provides users with an AI-assisted way to discover, evaluate, and hire verified skilled labour agencies across Bangladesh. By integrating 8-division geolocation search, dynamic multi-worker booking, a b500 dual-escrow deposit model, full bilingual accessibility (Bangla & English), and AI-driven budgeting and automated labour contracts, the platform promotes transparent, reliable, and safe hiring for both clients and artisans.

Future Work

- Multilingual & Regional Dialect Support
- Mobile Application (React Native Android & iOS)
- Real-time GPS Worker Tracking
- Biometric Government NID Verification API
- Blockchain Smart Contract Escrow System
- Bangla Voice-Based Search

References

1. Next.js Documentation (<https://nextjs.org/docs>)
2. Express.js Documentation (<https://expressjs.com/>)
3. MongoDB Atlas & Mongoose Documentation (<https://mongoosejs.com/>)
4. JWT (JSON Web Token) Documentation (<https://jwt.io/>)
5. Netlify Documentation (<https://docs.netlify.com/>)
6. Render Platform Documentation (<https://render.com/docs>)
7. React Documentation (<https://react.dev/>)
8. Bangladesh Bureau of Statistics (Labour Force Survey)